



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 10. Кейсы
(/teach/control/stream/view/id/931478155)

PErr10.3. Генератор видео. Часть 1 (new)

Предыдущий урок

(/pl/teach/control/lesson/view?id=346712532)

Следующий урок

(/pl/teach/control/lesson/view?id=341378470)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Что мы умеем

- ✓ Работаем с Proху API для доступа к нейросетям (OpenAI SORA, Google и др.) через единый интерфейс.
- ✓ Регистрируемся, пополняем баланс, создаем API-ключи и настраиваем .env-файлы.
- ✓ Генерируем видео по промπτ: отправляем запросы, отслеживаем статус с прогресс-баром, скачиваем MP4-файлы в Python.

Результат дня

Действия

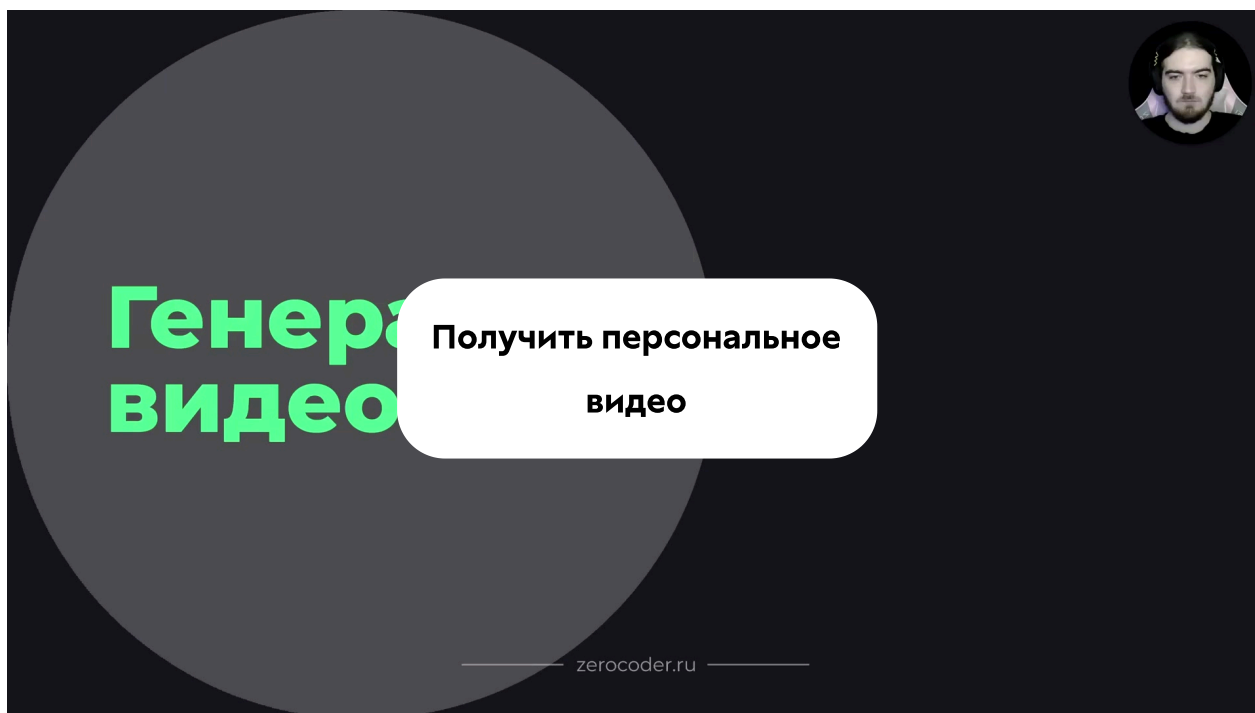
Узнаешь, как через Proxy API генерировать видео с помощью моделей вроде SORA: от настройки аккаунта и первого запроса до сохранения результата в базовом сервере для Telegram-бота и Flask-сайта.

▶ Время чтения и просмотра: ~ 1,5 часа

🕒 Время выполнения задания: ~ 45 минут

Теория на сегодня

1. Работа с API-платформами



Сегодня мы начинаем второй кейс курса — приложение для генерации видео, которое будет работать в **Telegram-боте** и на **Flask-сайте**. На этом уроке разберём базовый функционал и первый запрос, а в следующем подключим бота и веб-интерфейс.

Сегодня мы:

1. Разберем основной функциональный блок приложения, который будет ^{Действия} работать для пользователей.

2. Поймем, как отправлять запросы к различным нейросетям и почему нельзя ограничиваться одним провайдером.
3. Обсудим сервисы, которые позволяют работать с разными нейросетями через единую платформу и API.
4. Сделаем первый запрос на генерацию видео: от промпта до получения и скачивания результата.
5. Реализуем базовый функционал приёма и возврата данных через сервер-обработчик.
6. Обсудим, как дальше развивать приложение и почему в следующем уроке потребуется удобная пользовательская обертка.

▼ API-платформы

 Основная задача — создать скрипт для генерации видео.

 Цель урока — научиться работать с различными **API** через единый сервис.

Разберем работу с **API-платформами**, которые дают доступ к различным нейросетям. Существует множество провайдеров, которые предоставляют доступ к своим нейросетям через **API**. Это удаленные умные алгоритмы, которые работают на серверах провайдера, нужно просто отправлять им запросы и получать результат.

Таких сервисов сейчас очень много:

- **Google;**
- **OpenAI;**
- **YandexGPT;**
- **GigaChat;**
- **DeepSeek;**
- **Perplexity;**
- **Kling;**
- и многие другие.

⚠ У каждого из них свой **API**, свои модели и свои особенности.

Было бы неудобно работать с каждым провайдером отдельно.
Поэтому существуют **API-платформы-агрегаторы**.

✓ **API-агрегаторы** — платформы, которые дают единый доступ сразу к нескольким нейросетям через один интерфейс.

Примеры таких сервисов:

- Proxy API;
- GenAPI;
- и другие аналогичные платформы.

⚠ Все подобные сервисы работают по одному и тому же принципу. Научившись работать с одним **API-агрегатором**, можно без проблем перейти на любой другой.

Задача урока:

- понять принцип работы таких сервисов;
- научиться делать запросы;
- использовать ^{Действия} их для генерации видео.

Через такой **API** появляется возможность улучшать входной промпт для генерации видео. Пользователь передает обычный текстовый запрос, а система перед отправкой в модель может автоматически его обработать.

Например:

- перевести запрос на английский язык, так как большинство моделей лучше работают именно с ним;
- уточнить формулировки;
- структурировать запрос;
- убрать лишний шум и неточности.

 Заранее неизвестно, какие именно промпты будут вводить пользователи. Чтобы на выходе всегда получалось качественно сгенерированное видео, необходимо приводить входные данные к понятному и «чистому» виду.

Как и в любом алгоритме работы с данными: если входные данные некачественные — с лишней информацией, ошибками или плохой структурой — на выходе невозможно получить хороший результат. Плохой вход всегда дает плохой выход.

Использование подобного **API-сервиса** позволяет автоматически дорабатывать промпт до оптимального состояния, чтобы каждый пользователь независимо от опыта получал аккуратный и предсказуемый результат.

Тем же самым сервисом можно:

- генерировать изображения;
- работать с текстом;
- использовать TTS (text-to-speech);
- использовать speech-to-text;
- подключать другие функции, которые предоставляет рынок **AI-сервисов**.

⚠ **Отдельное преимущество** — доступ ко всем современным моделям. Это новые версии **GPT**, модели **OpenAI** и другие решения.

При этом, несмотря на ограничения прямого доступа к **OpenAI** в РФ, через такие **API-сервисы** с ними можно работать без обходных схем и дополнительной инфраструктуры.

⚠ Такой сервис — это отдельная платформа с собственной инфраструктурой, серверами и поддержкой. Поэтому каждый запрос обходится немного дороже. В среднем удорожание составляет около 5–10% от стоимости запроса.

Взамен пользователь получает:

- работу с российской платформой;
- отсутствие необходимости настраивать прокси или обходы;
- запросы на серверы в РФ;
- ответы от самых новых моделей, включая актуальные версии **GPT** или модели для генерации видео.

⚠ Но даже при наличии прямого аккаунта **OpenAI API-доступ** к таким моделям отсутствует. Делать прямые **API-запросы** невозможно — это ограничение самой платформы.

Единственный вариант работы с такими моделями — использование стороннего сервиса, который берет на себя взаимодействие с моделью, а пользователь работает уже через его **API**.

Преимуществ у такого подхода значительно больше. Единственный реальный недостаток — небольшое увеличение стоимости каждого запроса, которое компенсируется удобством, стабильностью и доступом к современным **AI-моделям**.

2. Обзор возможностей сервиса Proxy API

Возможности ProxyAPI



1. Доступ ко всем современным моделям

Одно API - множество функций

2. Работа с сервисами, которые не умеют работать напрямую

Возможность работать с OpenAI, GPT-4, GPT-3.5, GPT-3.5-turbo и тд

3. API доступ

Возможность работать с моделями по удобной API(набор удобных правил, инструментов и протоколов). Единый API для всех наших проектов

zerocoder.ru

▼ Возможности Proxy API

Рассмотрим возможности **Proxy API**. Важны три ключевые возможности.

1. API-доступ.



API — это самый удобный и универсальный способ взаимодействия между клиентом и сервером или между серверами.

Принцип **API-запросов** везде одинаковый: отправляется запрос, на стороне сервиса происходит обработка, возвращается ответ.



Основное преимущество **Proxy API** и аналогичных платформ — единый **API-доступ** ко множеству провайдеров. В одном проекте можно использовать модели **OpenAI**, в другом — **Google**, в третьем — другие библиотеки и сервисы. При этом все запросы проходят через один и тот же **API-интерфейс**.

Преимущества:

- Сильно упрощает работу.
- Нет необходимости создавать и поддерживать десятки аккаунтов у разных провайдеров.
Действия

- Достаточно одного аккаунта в **Proxy API**, через который можно в нужный момент выбирать подходящую модель или сервис.

2. Доступ ко всем современным моделям.

Один **API** дает доступ сразу к множеству функций:

- генерации текста;
- изображений;
- видео;
- аудио.

Это модели разных компаний — как уже проверенные временем, так и самые новые версии, включая актуальные релизы.



При наличии одного **API-доступа** можно использовать разные типы моделей в рамках одного проекта и легко переключаться между ними.

3. Работа с сервисами, которые не предоставляют прямой доступ.

Многие крупные платформы блокируют **API-запросы** или ограничивают функциональность при прямом подключении. Среди таких сервисов — **OpenAI**, **Google**, **11Labs** и другие.

Через **Proxy API** появляется возможность легально и стабильно работать с этими сервисами и использовать их алгоритмы для решения прикладных задач.

Важно понимать, что речь идет об одних из самых сильных инструментов на рынке:

- модели **OpenAI** — одни из лучших в работе с текстом, изображениями и видео;
- современные видеомодели, такие как **SORA** и ее обновленные версии, обеспечивают высокое качество генерации;
- модели **Google** отличаются высокой скоростью и точностью, а интеграция с экосистемой **Google** дает дополнительные преимущества, например, при работе с поиском;
- **11Labs** является абсолютным лидером в области работы с голосом.



Библиотеки 11Labs обеспечивают практически идеальное распознавание речи (speech-to-text) и генерацию речи (text-to-speech). Сгенерированные аудиофайлы звучат

максимально естественно: присутствуют паузы, интонации, запинки, характерные для живой человеческой речи. В большинстве случаев отличить такой голос от записанного человеком практически невозможно.

Все эти инструменты значительно превосходят большинство конкурентов и часто становятся основой для реальных коммерческих решений.

✓ **Proxy API** выступает как универсальное и удобное решение для работы с такими сервисами. Оно позволяет сосредоточиться на логике приложения и результате, а не на обходе ограничений и технических нюансах доступа.

Переходим к практике: создадим аккаунт, первичную настройку, познакомимся с интерфейсом и реализацией функциональной части приложения. В следующем уроке основной фокус будет на внешней обертке и пользовательском интерфейсе, чтобы с реализованным функционалом было удобно и понятно работать.

3. Регистрация на сервисе Proxy API и алгоритм начала работы

The screenshot shows the Proxy API website in a browser. The navigation bar includes links for 'Главная', 'Подписка Pro', 'OpenRouter', 'Тарифы', 'Контакты', and 'Документация'. A blue button labeled 'Вход' is in the top right corner. The main heading is 'Как это работает?'. Below it, a three-step process is outlined: 1. Register and get an API key, 2. Requests go through a chain of proxy servers in Europe to avoid regional blocking, 3. The service receives an answer from the provider and forwards it to the user's application. A large play button icon is centered over this text. Below the steps, the section 'Наши преимущества' (Our advantages) lists three benefits: 'Без блокировок и VPN' (No blocking and VPN), 'Без иностранного телефона' (No foreign phone), and 'Без карты иностранного банка' (No foreign bank card). Each benefit includes a brief explanation of how Proxy API handles the request. A small circular profile picture of a man with a beard is visible in the bottom left corner. The browser's taskbar at the bottom shows the Windows logo, search bar, and various application icons, with the system clock indicating 22:16 on 30.01.2026.

Как это работает?

1. Зарегистрируйтесь и получите ключ API. Замените адрес API в своем приложении на наш, методы и параметры останутся такими же, как у оригинального API, можно использовать официальные библиотеки
2. Ваши запросы будут отправляться через цепочку прокси-серверов, расположенных в Европе. Таким образом, мы исключаем вероятность блокировки по региональному принципу
3. Мы получаем ответ от поставщика услуг, который отправляем его через цепочку прокси-серверов, и передаем этот ответ вашему приложению

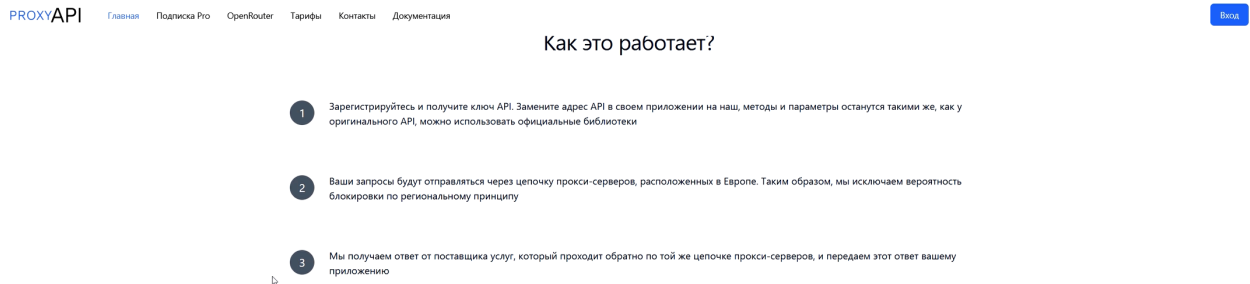
Наши преимущества

- Без блокировок и VPN
Ваши запросы будут отправляться через цепочку прокси-серверов, исключая вероятность блокировки
- Без иностранного телефона
Для регистрации не нужно иметь иностранный телефон, достаточно адреса электронной почты
- Без карты иностранного банка
Оплачивайте за потребляемые услуги в рублях с карты любого российского банка

Действия

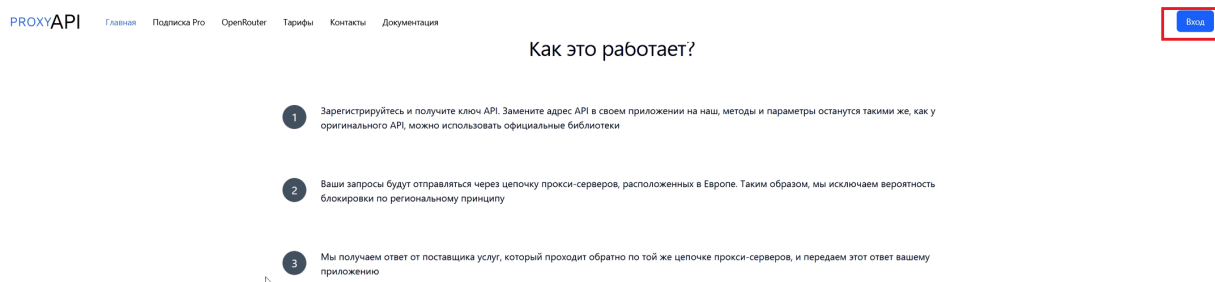
▼ Регистрация на Proxy API

После перехода на **Proxy API** открывается главная страница сервиса. Здесь можно ознакомиться с общей информацией: как работает платформа, какие возможности она предоставляет, какие преимущества есть у сервиса и для каких задач он подходит. На этом этапе полезно просто изучить сайт и понять общую логику работы платформы.

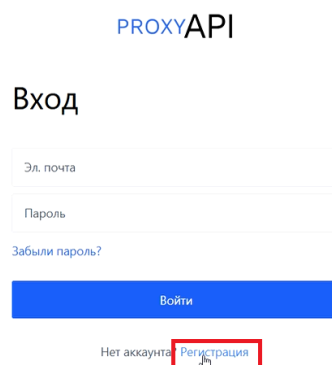


Чтобы начать работу, необходимо пройти регистрацию:

- В правом верхнем углу нажимаем кнопку **Вход**.



- Выбираем пункт **Регистрация**.



- В форме регистрации указываем электронную почту, имя, фамилию и пароль. Пароль вводится дважды для подтверждения.
- Соглашаемся с условиями использования сервиса, политикой обработки персональных данных и другими обязательными документами.
- Нажимаем кнопку **Зарегистрироваться**.

Действия

Регистрация

Эл. почта
Имя
Фамилия
Пароль
Подтверждение пароля

☐ Я согласен с условиями использования и политикой конфиденциальности .

Зарегистрироваться

Уже есть аккаунт? [Войти](#)

- После успешной регистрации выполняется вход в систему: вводится почта и пароль, и открывается панель управления сервиса.

Главная

Добро пожаловать в личный кабинет сервиса ProxyAPI!
В меню слева вы можете редактировать свой аккаунт, создавать и отзывать ключи API и просматривать статистику использования и платежей.
На этой странице мы будем публиковать новости и обновления сервиса.

⚠ Бесплатных вариантов генерации видео и изображений не существует. Бесплатные лимиты для подобных задач отсутствуют у всех сервисов без исключения.

Генерация изображений и особенно видео является крайне ресурсоемким процессом. Генерация видео требует значительных вычислительных мощностей.

Если рассматривать принцип работы языковых моделей, он основан на большом количестве сложных математических операций с векторами. Для текста таких операций относительно немного.

В случае генерации изображений и видео принцип остается тем же, но масштаб вычислений увеличивается в разы. Для видео количество обрабатываемых векторов не просто больше, а может превышать текстовые задачи в десятки тысяч раз. Это создает высокую нагрузку на серверную инфраструктуру.

⚠ Каждый запрос на генерацию видео обходится сервису дорого: требуется мощное оборудование, высокая пропускная способность и значительные затраты на обработку данных.

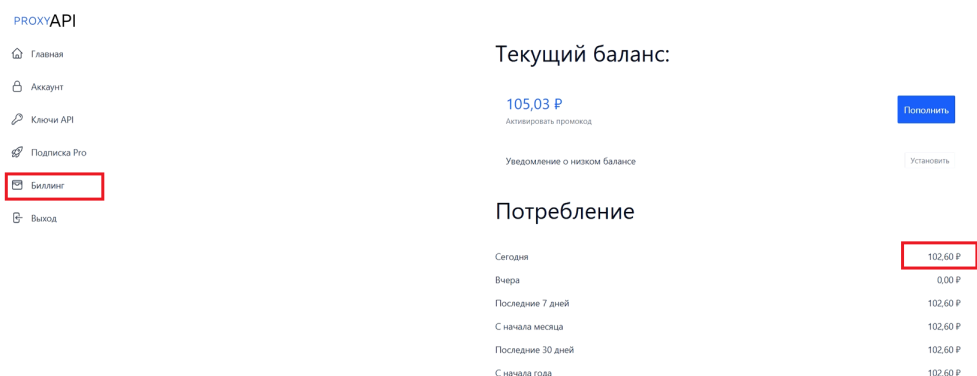
▼ Алгоритм начала работы с Proxy API

Процесс начала работы является универсальным. Так работают не только **Proxy API**, но и большинство аналогичных сервисов.

Независимо от платформы, алгоритм взаимодействия всегда примерно одинаковый:

1. Пополнение баланса:

- Переходим в раздел Биллинг .



PROXYAPI

- Главная
- Аккаунт
- Ключи API
- Подписка Pro
- Биллинг**
- Выход

Текущий баланс:

105,03 R

Активировать промокод

Уведомление о низком балансе

Пополнить

Установить

Потребление

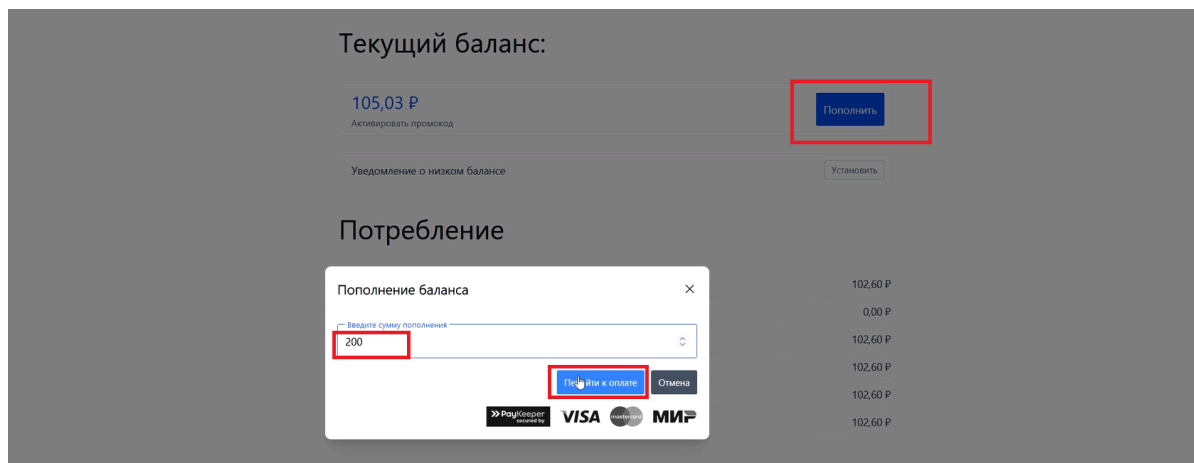
Сегодня	102,60 R
Вчера	0,00 R
Последние 7 дней	102,60 R
С начала месяца	102,60 R
Последние 30 дней	102,60 R
С начала года	102,60 R

В этом разделе отображается текущий баланс и история списаний. По фактическим данным можно ориентироваться на стоимость генерации одного видео.

В среднем генерация одного короткого видео обходится примерно в 100–105 рублей. Например, около 102 рублей за ролик длительностью около 4 секунд при использовании одной из самых доступных по цене моделей. Эти значения можно использовать как ориентир при расчете бюджета проекта.

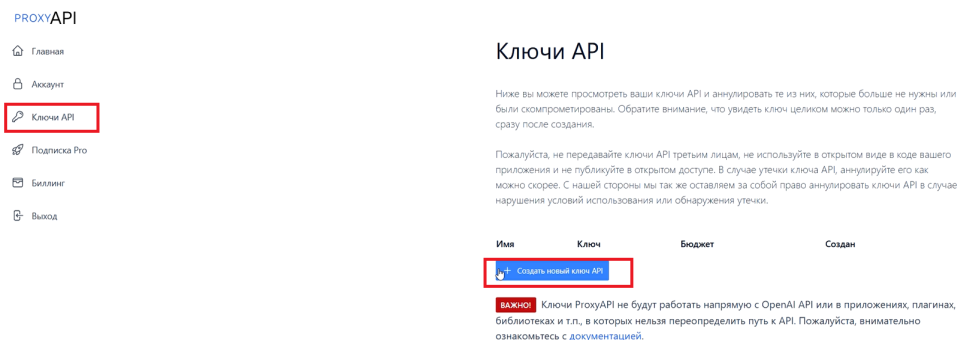
⚠ Минимальная сумма пополнения в **Proxy API** составляет 200 рублей. Если попытаться ввести меньшую сумму, система не позволит продолжить оплату.

- Нажимаем Пополнить → вводим сумму → нажимаем Перейти к оплате → оплата производится стандартными способами — через СБП, банковские карты и другие доступные методы.

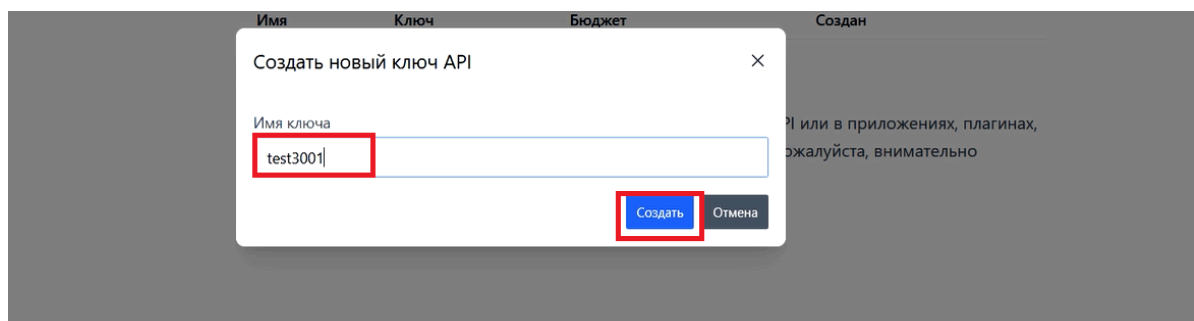


2. Создание API-ключа.

- Переходим в раздел Ключи API → нажимаем Создать новый ключ API.



- Задаем название ключа (например, тестовое имя с датой создания) → нажимаем Создать.

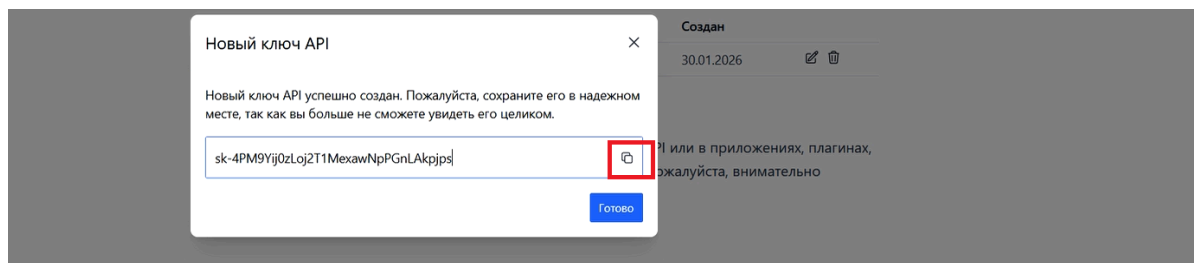


⚠ Каждый ключ показывается пользователю только один раз — в момент создания. Повторно посмотреть этот ключ через интерфейс сервиса невозможно. При необходимости можно создать новый ключ или удалить старый, но восстановить уже созданный ключ нельзя. Это сделано из соображений безопасности, чтобы исключить утечку доступа к **API**.

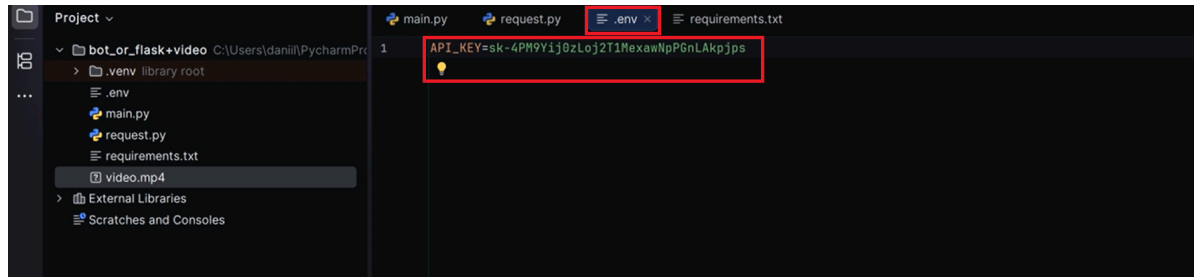
3. Настройка ключа.

- После генерации ключ копируем.

Действия



- Переходим в проект → добавляем в файл переменных окружения (`.env`).

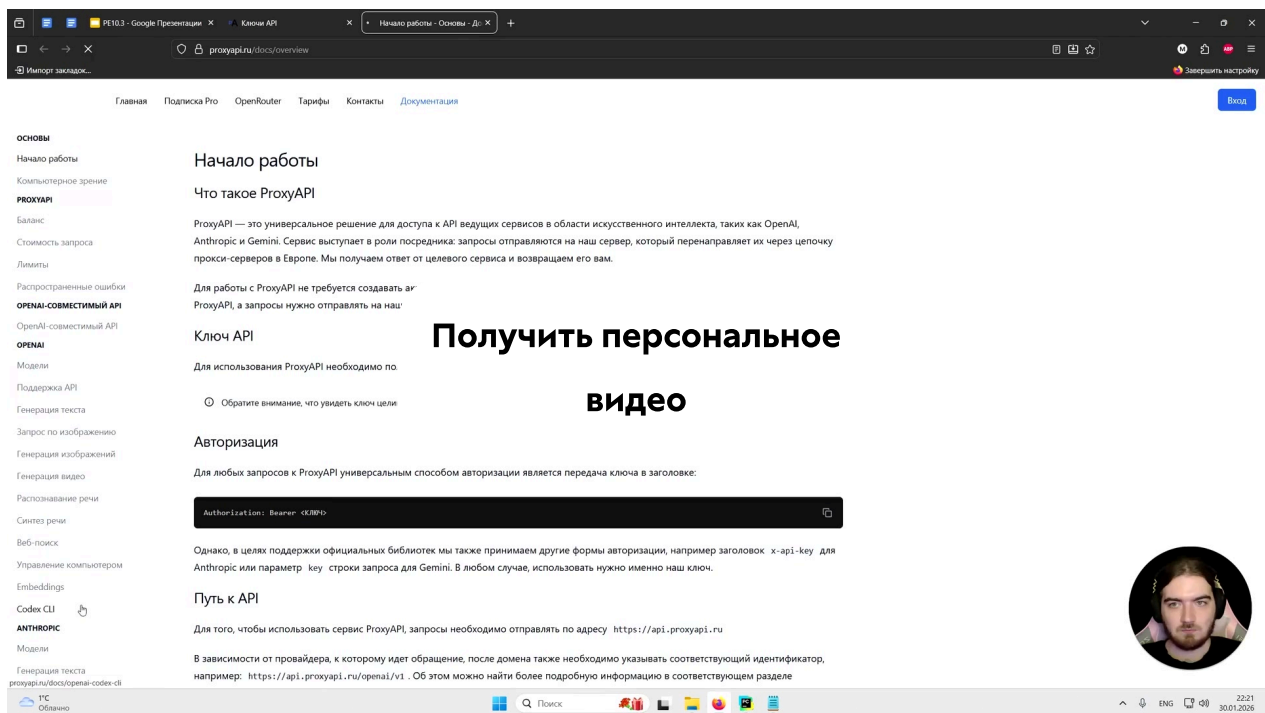


- ✓ Переменные окружения используются для хранения конфиденциальных данных: **API-ключей**, токенов доступа и других чувствительных параметров.

Сохранение ключа на этом этапе критически важно, так как повторно получить его через сайт не получится.

После этого настройка ключа считается завершенной. Даже при редактировании или просмотре параметров ключ больше не отображается в явном виде.

4. Работа с документацией сервиса Proxu API и генерация первого видео

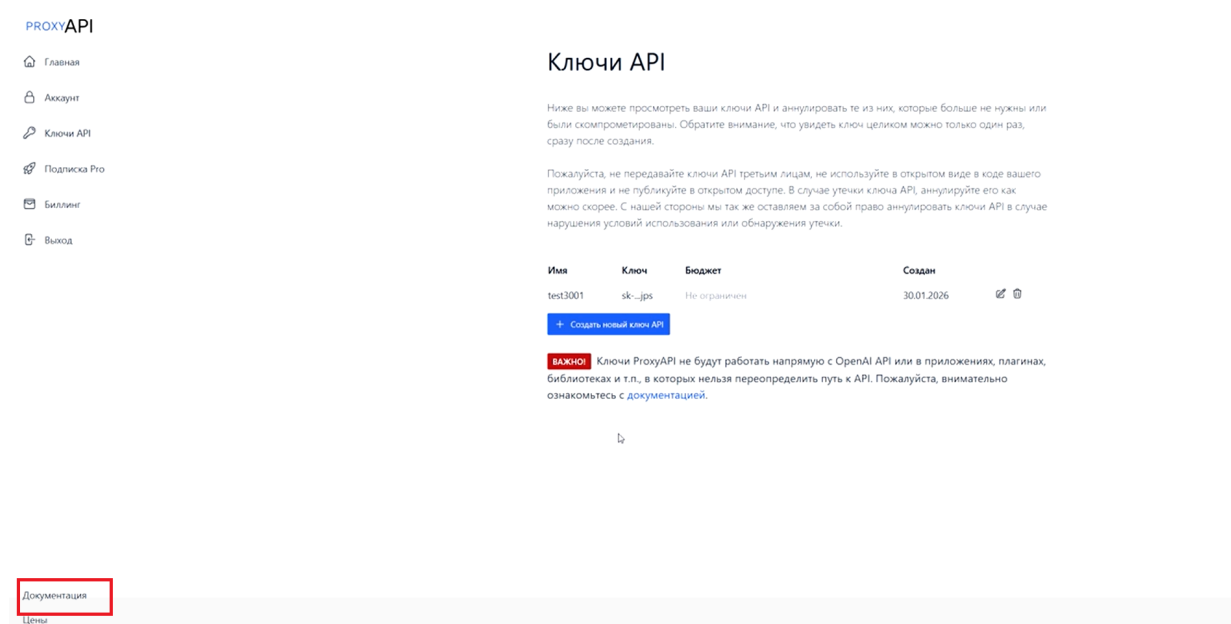


▼ Изучение документации

Работа с документацией сервиса — обязательный шаг, так как разные платформы и провайдеры подключаются по-разному: где-то используются обычные **HTTP-запросы**, где-то готовые **SDK**, например библиотека **OpenAI** или **Google**.

 В любом случае работа всегда начинается с изучения документации.

1. Переходим в раздел Документация .



2. Выбираем вкладку Генерация видео → изучаем раздел Начало работы , где можно найти общее описание возможностей сервиса и оглавление.

Отдельно смотрим раздел **OpenAI** и генерация видео с помощью модели **SORA**.

PROXYAPI

ГлавнаяПодписка ProOpenRouterТарифыКонтактыДокументация

Распространенные ошибки

OPENAI-СОВМЕСТИМЫЙ API

OpenAI-совместимый API

OPENAI

Модели

Поддержка API

Генерация текста

Запрос по изображению

Генерация изображений

Генерация видео

Распознавание речи

Синтез речи

Веб-поиск

Управление компьютером

Embeddings

Codemx CLI

ANTHROPIC

Начало работы

Что такое ProхуAPI

ProхуAPI — это универсальное решение для доступа к API ведущих сервисов в области искусственного интеллекта, таких как OpenAI, Anthropic и Gemini. Сервис выступает в роли посредника: запросы отправляются на наш сервер, который перенаправляет их через цепочку прокси-серверов в Европе. Мы получаем ответ от целевого сервиса и возвращаем его вам.

Для работы с ProхуAPI не требуется создавать аккаунты в сторонних системах — управление и оплата происходит через ваш личный кабинет ProхуAPI, а запросы нужно отправлять на наши серверы.

Ключ API

Для использования ProхуAPI необходимо получить ключ. Для этого необходимо зарегистрироваться на сайте и перейти в раздел Ключи API.

Обратите внимание, что увидеть ключ целиком можно только один раз, сразу после создания.

Авторизация

Для любых запросов к ProхуAPI универсальным способом авторизации является передача ключа в заголовке:

 **SORA** — это одна из самых современных разработок **OpenAI** в области генеративных медиа.

Видео **API** предоставляет несколько эндпоинтов, каждый из которых отвечает за отдельный этап работы с видео.

PROXYAPI

ГлавнаяПодписка ProOpenRouterТарифыКонтактыДокументация

Распространенные ошибки

OPENAI-СОВМЕСТИМЫЙ API

OpenAI-совместимый API

OPENAI

Модели

Поддержка API

Генерация текста

Запрос по изображению

Генерация изображений

Генерация видео

Распознавание речи

Синтез речи

Веб-поиск

Управление компьютером

Embeddings

Codemx CLI

Генерация видео с помощью Sora

Создавайте, редактируйте и управляйте видео с помощью Sora API.

Обзор

Sora — это новейшая разработка OpenAI в области генеративных медиа: современная модель для создания видео, способная генерировать детализированные динамические ролики со звуком на основе текстового описания или изображений. Построенная на многолетних исследованиях мультимодальной диффузии и обученная на разнообразных визуальных данных, Sora обладает глубоким пониманием 3D-пространства, движения и непрерывности сцен.

Video API предоставляет пять эндпоинтов, каждый из которых имеет свои возможности:

- Создание видео: запуск нового задания рендеринга из промпта с опциональными референсными изображениями или ID ремикса
- Получение статуса видео: проверка текущего состояния задания и мониторинг его прогресса
- Скачивание видео: получение готового MP4-файла после завершения задания
- Список видео (не поддерживается в ProхуAPI): просмотр ваших видео с пагинацией для истории или управления
- Удаление видео: удаление отдельного видео по ID из хранилища OpenAI

Основные эндпоинты:

- создание видео;
- получение статуса генерации;
- скачивание готового видео;
- получение списка видео;
- удаление видео.

Используем три ключевых эндпоинта: создание видео, получение статуса генерации, скачивание готового результата.

Действия



Генерация видео — это дорогой и длительный процесс.

Видео не создается мгновенно. Сначала отправляется запрос на запуск генерации, после чего создаётся задача. Эта задача некоторое время находится в очереди, затем обрабатывается, и только после завершения становится доступен готовый результат.

Именно поэтому логика работы всегда строится в несколько шагов:

- отправка запроса на генерацию;
- периодическая проверка статуса задачи;
- загрузка готового видео после завершения генерации.



На выходе модель не сразу возвращает файл в формате **MP4**. Сначала формируется внутреннее представление — результат работы модели в виде закодированных данных. Далее выполняется этап рендеринга — преобразование этих данных в полноценный видеофайл, который уже можно скачать и воспроизвести.

Получение статуса генерации необходимо для того, чтобы понять, завершен ли процесс или видео все еще находится в очереди или на этапе обработки. После статуса **Готово** становится доступна загрузка итогового файла.

3. Пролистываем документацию ниже → находим приведенный пример запуска генерации видео с использованием текстового промпта и дополнительных параметров.

Запуск задания рендеринга

Начните с вызова `POST /videos` с **текстовым промптом и необходимыми параметрами**. Промпт определяет творческий вид и настроение — объекты, камеру, освещение и движение, — в то время как параметры вроде `size` и `seconds` контролируют разрешение и длительность видео.

curl python javascript

```
from openai import OpenAI

client = OpenAI(
    api_key="sk-K00t5",
    base_url="https://api.proxyapi.ru/openai/v1",
)

video = client.videos.create(
    model="sora-2",
    prompt="Видео с надписью 'Спасибо' сверкающими буквами",
)

print("Генерация видео началась:", video)
```

Ответ представляет собой JSON-объект с уникальным `id` и начальным статусом, таким как `queued` или `in_progress`. Это означает, что задание рендеринга запущено.

Действия



Промпт определяет визуальный стиль, атмосферу, объекты, движение камеры и освещение.

Еще ниже располагается формат ответа, что будем получать, ограничение правила, эффективные промпты, мониторинг.

Ответ представляет собой JSON-объект с уникальным `id` и начальным статусом, таким как `queued` или `in_progress`. Это означает, что задание рендеринга запущено.

```
{
  "id": "video_68d7512d07848190b3e45da0ecbebcde004da08e1e0678d5",
  "object": "video",
  "created_at": 1758941485,
  "status": "queued",
  "model": "sora-2-pro",
  "progress": 0,
  "seconds": "8",
  "size": "1280x720"
}
```

Ограничения и правила

4. Находим раздел с кодом.

Скачивание MP4

Когда задание достигнет статуса `completed`, получите MP4-файл через `GET /videos/{video_id}/content`. Этот эндпоинт передаёт бинарные данные видео и возвращает стандартные заголовки контента, поэтому вы можете либо сохранить файл напрямую на диск, либо передать его в облачное хранилище.

curl python javascript

```
curl -L "https://api.proxyapi.ru/openai/v1/videos/video_abc123/content" \
-H "Authorization: Bearer <КЛЮЧ>" \
--output video.mp4
```

Теперь у вас есть готовый видеофайл для воспроизведения, редактирования или распространения. URL для скачивания действителен максимум 1 час после генерации. Если вам нужно долгосрочное хранение, скопируйте файл в вашу собственную систему хранения незамедлительно.

Копируем его.

curl python javascript

```
from openai import OpenAI
import sys
import time

client = OpenAI(
    api_key="<КЛЮЧ>",
    base_url="https://api.proxyapi.ru/openai/v1",
)

video = client.videos.create(
    model="sora-2",
    prompt="Видео крутого кота на мотоцикле ночью",
)

print("Генерация видео началась:", video)

progress = getattr(video, "progress", 0)
bar_length = 30

while video.status in ("in_progress", "queued"):
    video = client.videos.retrieve(video.id)
    progress = getattr(video, "progress", 0)

    filled_length = int((progress / 100) * bar_length)
    bar = "-" * filled_length + "." * (bar_length - filled_length)
    status_text = "В очереди" if video.status == "queued" else "Обработка"

    sys.stdout.write(f"\r{status_text}: [{bar}] {progress:1f}%")
    sys.stdout.flush()
    time.sleep(2)

sys.stdout.write("\n")

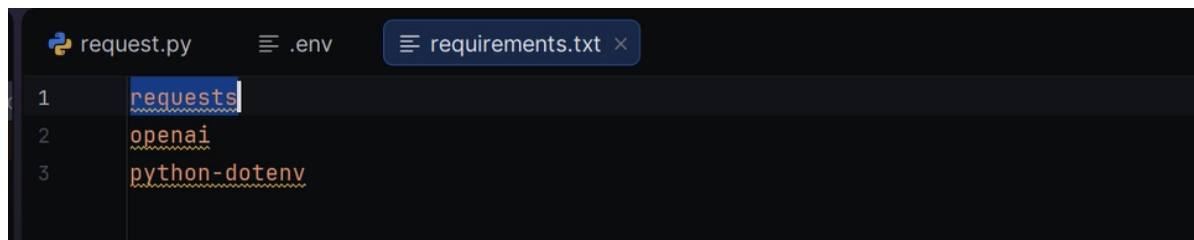
if video.status == "failed":
    message = getattr(
        getattr(video, "error", None), "message", "Генерация видео не удалась"
    )
    print(message)
else:
    print("Генерация видео завершена:", video)
    print("Скачивание видео...")

    content = client.videos.download_content(video.id, variant="video")
    content.write_to_file(f"video.mp4")
```

5. Переходим в PyCharm.:

- Вставляем скопированный код. Действия

- В проекте уже создан файл `.env` и основной файл `main.py`. Также присутствует файл `requests.py`, который в текущей реализации не используется, поэтому его можно удалить, чтобы не создавать путаницу.
- В файле `requirements.txt` перечислены библиотеки, необходимые для работы проекта:
`requests` — для выполнения HTTP-запросов,
`openai` — для работы с API в соответствии с документацией,
`python-dotenv` — для загрузки переменных окружения из файла `.env`.



```
1 requests
2 openai
3 python-dotenv
```

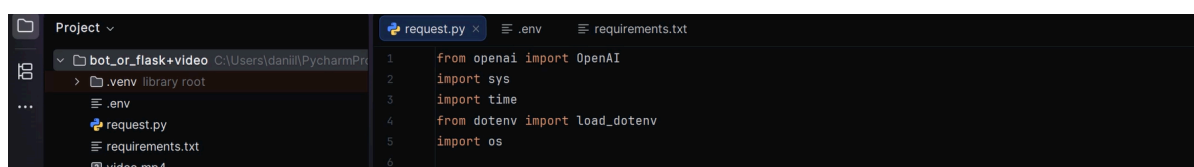
- Работа с **API** осуществляется через библиотеку `openai`. Несмотря на то, что **OpenAI** изначально предоставляет прямой доступ к своим серверам, библиотека позволяет указывать альтернативный эндпоинт. В этом случае запросы фактически отправляются не напрямую в **OpenAI**, а через **Proxy API**.

С точки зрения кода и логики работы ничего не меняется: используется стандартный интерфейс библиотеки **OpenAI**, но в качестве сервера указывается **Proxy API**. Это позволяет сохранить привычный подход к разработке и при этом обойти региональные ограничения.

- Файл `.env` используется для хранения чувствительных данных: **API-ключей**, токенов и других секретов. Эти данные не должны попадать в код и тем более в публичные репозитории.
- Все ключи хранятся в переменных окружения и подгружаются в коде автоматически через `python-dotenv`.

6. Рассмотрим обновленный код:

- В коде добавлены две библиотеки: `dotenv` и `os`. Их назначение — безопасно загружать переменные окружения из файла `.env`, чтобы токены и ключи **API** не хранились напрямую в коде и не могли быть скомпрометированы.



```
1 from openai import OpenAI
2 import sys
3 import time
4 from dotenv import load_dotenv
5 import os
```

Действия

- Вся логика работы с **API** помещена в отдельную функцию. Это позволяет в будущем импортировать ее в любой другой проект, например, в **Telegram-бот**, и использовать для генерации видео.
- Внутри функции создаем клиент **OpenAI** с указанием **API-ключа** и базового **URL** для запросов через **Proxy API** → вызываем эндпоинт `create` для видео.

```
from openai import OpenAI
import sys
import time
from dotenv import load_dotenv
import os

load_dotenv()

def generate_video(prompt): 1 usage
    client = OpenAI(
        api_key=f"{os.getenv('API_KEY')}",
        base_url="https://api.proxyapi.ru/openai/v1",
    )
```

- Выбираем модель для генерации: переходим в Документацию → Тарифы → используем **SORA 2**. Причина выбора — баланс между качеством и стоимостью: **SORA 2 Pro** дает чуть лучшее качество, но стоит почти в два раза дороже за секунду видео.

PROXYAPI Главная Подписка Pro OpenRouter **Тарифы** Контакты Документация

Генерация видео

Создают видеоролики на основе текстовых описаний. Применяются в маркетинге, обучении, развлечениях и кинопроизводстве.

Цена указана за одну секунду стенированного видео и отличается в зависимости от разрешения для некоторых моделей.

Модель	Разрешение	Цена за секунду
sora-2 от openai	720x1280 или 1280x720	27 P
sora-2-pro от openai	720x1280 или 1280x720	69 P
	1024x1792 или 1792x1024	106 P
veo-3 от google	Генерация 1 секунды видео	95 P
veo-3-fast от google	Генерация 1 секунды видео	43 P

- Дополнительно задаем параметры: `prompt` — текстовый запрос, описывающий, что должно быть сгенерировано и `seconds` — длительность видео. Для тестирования обычно достаточно 1–2 секунд. Копируем и вставляем в код.

Запуск задания рендеринга

Начните с вызова `POST /videos` с текстовым промптом и необходимыми параметрами. Промпт определяет творческий вид и настроение — объекты, камеру, освещение и движение, — в то время как параметры вроде `size` и `seconds` контролируют разрешение и длительность видео.

- Код проверяет корректность формата и типов параметров. Если появляются предупреждения, это не обязательно ошибка, а лишь сигнал для проверки типа данных. В результате при вызове функции создается видео по заданному промпту, с указанной моделью и длительностью, готовое для дальнейшей обработки или скачивания.

Действия

```

video = client.videos.create(
    model="sora-2",
    prompt=f"{prompt}",
    seconds="1",
)

```

- Запускаем процесс генерации видео, начинаем генерацию видео и выводим сообщение: «Генерация видео началась» → создаем прогресс-бар, который будет постепенно заполняться по мере генерации.

The screenshot shows a code editor with Python code for video generation and a terminal window displaying the output. The code includes a function to create a video and a loop to monitor its progress, updating a progress bar. The terminal output shows the video being generated and the progress bar being updated.

```

21 print("Генерация видео началась:", video)
22
23 progress = getattr(video, "progress", 0)
24 bar_length = 30
25
26 while video.status in ("in_progress", "queued"):
27     video = client.videos.retrieve(video.id)
28     progress = getattr(video, "progress", 0)
29
30     filled_length = int((progress / 100) * bar_length)
31     bar = "=" * filled_length + "-" * (bar_length - filled_length)
32     status_text = "В очереди" if video.status == "queued" else "Обработка"

```

Terminal output:

```

(.venv) PS C:\Users\daniil\PycharmProjects\lessons\bot_or_flask+video> python .\request.py
Генерация видео началась: Video(id='video_697d00fd58788190bcc310f3c0923c4706b0ca330a9fc696', completed_at=None, created_at=1769799933, error=None, expires_at=None, model='sora-2', object='video', progress=0, prompt='Крупный план чашки горячего кофе на деревянном столе, утренний свет сквозь жалюзи, мягкая глубина резкости.', remixed_from_video_id=None, seconds='4', size='720x1280', status='queued')
Обработка: [=====] 100.0%
Генерация видео завершена: Video(id='video_697d00fd58788190bcc310f3c0923c4706b0ca330a9fc696', completed_at=1769799991, created_at=1769799933, error=None, expires_at=1769799991, object='video', progress=100, prompt='Крупный план чашки горячего кофе на деревянном столе, утренний свет сквозь жалюзи, мягкая глубина резкости.', remixed_from_video_id=None, seconds='4', size='720x1280', status='completed')
Скачивание видео...
Файл video.mp4 сохранён

```

- Прогресс-бар сначала тестируем в терминале, в дальнейшем планируем интегрировать на сайт и в **Telegram-бота**, чтобы пользователю было удобно и красиво отслеживать процесс.
- Отслеживаем прогресс видео: каждые 2–5 секунд проверяем, сколько уже сгенерировано видео → используем статус видео: `in_progress` — генерация идет; `queued` — видео в очереди на генерацию. Выводим прогресс в процентах, визуально обновляя прогресс-бар.

```

while video.status in ("in_progress", "queued"):
    video = client.videos.retrieve(video.id)
    progress = getattr(video, "progress", 0)

    filled_length = int((progress / 100) * bar_length)
    bar = "=" * filled_length + "-" * (bar_length - filled_length)
    status_text = "В очереди" if video.status == "queued" else "Обработка"

    sys.stdout.write(f"\r{status_text}: [{bar}] {progress:.1f}%")
    sys.stdout.flush()
    time.sleep(2)

```

- Обрабатываем ошибки: после завершения генерации проверяем статус видео. Если `failed` — выводим сообщение о неудачной генерации: «Генерация видео не удалась». Если ошибок нет — продолжаем процесс.
- Скачиваем готовое видео: когда генерация завершена (`status == complete`), используем метод `Download Content`, чтобы скачать видео по **ID**. Сохраняем видео локально в файл `video.mp4`. Выводим сообщение: «Файл `video.mp4` сохранен».

```
if video.status == "failed":
    message = getattr(
        getattr(video, "error", None), "message", "Генерация видео не удалась"
    )
    print(message)
else:
    print("Генерация видео завершена:", video)
    print("Скачивание видео...")

    content = client.videos.download_content(video.id, variant="video")
    content.write_to_file("video.mp4")

    print("Файл video.mp4 сохранён")

generate_video("Крупный план чашки горячего кофе на деревянном столе, утренний свет сквозь жалюзи, мягкая глубина резкости.")
```

- Тестируем работу скрипта, используем промпт:

 *Крупный план чашки горячего кофе на деревянном столе, утренний свет сквозь жалюзи, мягкая глубина резкости.*

- Запускаем скрипт и наблюдаем: генерация видео началась, отображается ID и прогресс; после завершения видео статус меняется на complete ; файл видео успешно сохраняется.

```
(.venv) PS C:\Users\danil\PycharmProjects\lessons\bot_or_flask+video> python .\request.py
Генерация видео началась: Video(id='video_697d08fd5878819b0cc310f3c0923c4706b0ca330a9fc696', completed_at=None, created_at=1769799933, error=None, expires_at=None, model='sora-2', object='video', progress=0, prompt='Крупный план чашки горячего кофе на деревянном столе, утренний свет сквозь жалюзи, мягкая глубина резкости.', remixed_from_video_id=None, seconds='4', size='720x1280', status='queued')
Обработка: [=====] 100.0%
```

- В терминале видно детали: время создания, ошибки (если есть), когда видео будет удалено, использованная модель, прогресс и статус.

```
Генерация видео завершена: Video(id='video_697d08fd5878819b0cc310f3c0923c4706b0ca330a9fc696', completed_at=1769799991, created_at=1769799933, error=None, expires_at=1769803591, model='sora-2', object='video', progress=100, prompt='Крупный план чашки горячего кофе на деревянном столе, утренний свет сквозь жалюзи, мягкая глубина резкости.', remixed_from_video_id=None, seconds='4', size='720x1280', status='completed')
Скачивание видео...
Файл video.mp4 сохранён
```

- В начале генерации прогресс 0%, статус «в очереди». После завершения процесса прогресс достигает 100%, статус становится complete . Видео доступно для скачивания и дальнейшей обработки.
- Запускаем скрипт с генерацией видео и проверяем, как работает ограничение по количеству секунд. Получаем ошибку: минимальная длительность — 4 секунды, поддерживаются варианты: 4, 8, 12 секунд.
- Выбираем минимальное время 4 секунды и запускаем снова.

```
video = client.videos.create(
    model="sora-2",
    prompt=f"{prompt}",
    seconds="4",
```

- Генерация видео началась. Видим модель и процесс обработки, прогресс по мере создания видео (например, 10%). При необходимости удаляем старое видео, чтобы новый файл точно сохранился.

 Генерация видео — не мгновенный процесс, нужно немного подождать.
Действия

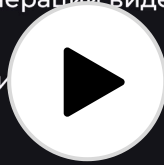
- После завершения прогресса (99–100%) видео собирается в MP4. Возможна ошибка запроса (например, блокировка), часто связанная с некорректным первым запросом или лимитом средств.
- Создаем файл `test.py` для проверки статуса видео по `video.id`. Запрашиваем статус через **API**, чтобы убедиться, что видео обработано корректно. Подтверждаем, что ложных списаний средств не произошло: средства списаны только за корректные запросы.
- Сравниваем с предыдущим видео: новое видео отличается по кадрам, но соответствует промπτу. Генерация выглядит реалистично, видео практически неотличимо от съемки вживую.



5. Итоги урока

Итоги урока

1. Изучили как мы можем делать запросы к различным нейросетям
2. Сделали первый запрос для генерации видео по промπτу
3. Решили как будем улучшать при



zerocoder.ru



► Результаты урока

Дополнительные материалы

- Код проекта эксперта 
- Ссылки на программы и сайты

Домашнее задание

1. Доработай скрипт для генерации видео через Proxy API, используя собственный промπτ (описание видео, отличное от примера эксперта с чашкой кофе).
2. Протестируй генерацию с длительностью 4 секунды, проверь статус через test.py и сохрани MP4-файл.

В качестве домашнего задания пришли скриншоты:

- написанного кода с твоим промπτом;
- терминала с прогресс-баром и статусом (queued → complete);

Действия

- сгенерированного видео (первые/последние кадры или QR-код для скачивания).

На следующем занятии

На следующем уроке мы обернем базовый генератор видео в удобные пользовательские интерфейсы: создадим веб-сайт на Flask и Telegram-бота для приема промптов от пользователей и возврата готовых MP4-файлов.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

PErr10.3. Генератор видео. Часть 1 (new)

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

1. Доработай скрипт для генерации видео через Proxy API, используя собственный промпт (описание видео, отличное от примера эксперта с чашкой кофе).
2. Протестируй генерацию с длительностью 4 секунды, проверь статус через test.py и сохрани MP4-файл.

В качестве домашнего задания пришли скриншоты:

- написанного кода с твоим промптом;
- терминала с прогресс-баром и статусом (queued → complete);
- сгенерированного видео (первые/последние кадры или QR-код для скачивания).



Ваш ответ



Прикрепить файлы

максимальный размер файла - 100мб

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Действия

Предыдущий
урок

(<https://university.zerocoder.ru/pl/teach/control/lesson/view?id=346712532>)

Следующий урок

Действия